FREE RESOURCE

THE CLAUDE CONTENT SYSTEM

Make your app visible in the age of AI. Rank first on Google and get recommended by AIs like Claude and ChatGPT.

Copy-paste prompts to transform your blog SEO, run keyword research, automate publishing, and create a reusable blog writer skill — all with Claude Code.

WHAT'S INSIDE

1. **The Content Machine Prompt**
   Turn any Next.js blog into a fully SEO & LLM-optimized content machine
2. **The SEMrush Research Prompt**
   Run a full keyword strategy analysis using the Semrush MCP
3. **The Blog Writer Skill**
   A reusable Claude Code skill that writes SEO blog posts on autopilot
4. **The Daily Blog Automation Setup**
   Automate daily blog publishing with Claude Code scheduling

PROMPT 1 OF 4

THE CONTENT MACHINE PROMPT

Copy this entire prompt and paste it into Claude Code in your Next.js project. It transforms a basic blog into a fully SEO and LLM-optimized content machine.

PASTE INTO CLAUDE CODE

I need you to transform this Next.js project into an SEO-optimized, LLM-discoverable content machine. Do everything step by step — don't skip anything.

1. Audit the current project

Read the project structure and identify:

- All existing pages and routes
- The blog system (how posts are stored, rendered, and routed)
- Any existing meta tags, Open Graph tags, or structured data
- The current sitemap situation (does one exist?)
- The current robots.txt situation

Tell me what you found before proceeding.

2. Create or fix the sitemap

Create/update `app/sitemap.ts` that:

- Auto-generates entries for ALL pages (static + blog posts)
- Sets proper changeFrequency and priority values
- Blog index gets priority 0.9, individual posts get 0.8
- Homepage gets priority 1.0, other static pages get 0.7
- Uses the site's canonical URL from a config file

3. Create or fix robots.txt

Create `app/robots.ts` with proper rules allowing all user agents and pointing to the sitemap.

4. Add JSON-LD structured data

- **Homepage:** Organization + WebSite schema
- **Each blog post:** Article schema with headline, description, author, datePublished, url, and keywords
- Create `lib/schema.ts` with helper functions, inject via `<script type="application/ld+json">`

5. Fix meta tags on every page

Every page needs in its metadata:

- **title** — unique, includes primary keyword
- **description** — under 160 chars, includes keyword, compelling
- **alternates.canonical** — the canonical URL
- **openGraph** — title, description, url, siteName, type, locale
- **twitter** — card type, title, description
- **keywords** — array of relevant terms
- **robots** — index: true, follow: true

6. Create LLM discoverability files

- Create `public/llms.txt` — short summary with topics, audience, and canonical URL
- Create `public/llms-full.txt` — full reference listing every article URL, every page, and contact info
- **IMPORTANT:** Every time you add a new blog post, update llms-full.txt with the new article URL and title. This is how LLMs discover new content.

7. Ensure static pre-rendering

- Blog post pages use `generateStaticParams` to pre-build at build time
- The output HTML contains full article text (not loaded client-side)
- Crawlers (Google, Bing, LLM crawlers) see complete content without JavaScript

8. Add internal linking infrastructure

- A "Related Articles" component at the bottom of each post (auto-calculated from tags/category)
- Each article should have 3-5 internal links to other articles placed naturally in the text

9. Set up the blog post template

Every blog post page should automatically include: navbar, post header (title, author, date, category), table of contents sidebar, article body, related articles, footer, JSON-LD Article schema, and full SEO meta tags.

10. Verify everything works

Run `npm run build` and confirm: all pages generate successfully, sitemap includes all routes, no build errors.

Then tell me: how many pages are in the sitemap, which pages have structured data, whether llms.txt and llms-full.txt are accessible, and the build status.

After you've done all of this, I'll connect Semrush MCP and we'll do keyword research to plan what content to write.

PROMPT 2 OF 4

THE SEMRUSH RESEARCH PROMPT

Use this after connecting the Semrush MCP. Paste this into Claude Code to run a complete keyword strategy analysis.

PASTE INTO CLAUDE CODE

I've connected the Semrush MCP to this project. I need you to do a full SEO research analysis for my website.

Step 1: Analyze my domain

Use the Semrush MCP tools to pull:

- Domain overview (organic keywords, traffic, rank)
- Current organic keywords we rank for
- Our competitors in organic search

Tell me what you find.

Step 2: Research keyword opportunities

Based on my site's topic and audience, research keywords across these categories:

1. **Core topic keywords**
   the main things we teach/cover
2. **Comparison keywords**
   "[our tool] vs [competitor]" type searches
3. **How-to keywords**
   "how to use [tool] for [task]"
4. **Role-based keywords**
   "[tool] for [job title]"
5. **Pricing/evaluation keywords**
   "is [tool] free", "[tool] pricing"

For each keyword, pull: search volume, keyword difficulty (KD), and CPC (shows commercial value).

Step 3: Build a keyword strategy

Organize the keywords into:

- **Priority 1 (write first):** KD 0-25, decent volume, strong fit
- **Priority 2 (write next):** KD 25-45, higher volume
- **Priority 3 (long-term):** KD 45+, aspirational

Step 4: Create the keyword cluster map

Group keywords into 3-4 clusters with hub pages and supporting pages:

```
Cluster 1: [Name]
  Hub page: [main article]
  Supporting: [article 1], [article 2], [article 3]
```

Step 5: Create the publishing plan

Order the first 10-15 articles by priority. For each: article title, target keyword, search volume and KD, cluster it belongs to, and page type (guide, comparison, template page, etc.).

Step 6: Save everything

Save the research to `references/seo-research/`:

- **keyword-strategy.md** — the full analysis
- **cluster-map.md** — the cluster structure
- **publishing-plan.md** — the ordered content queue

These files will be used by the blog writing skill as reference material.

PROMPT 3 OF 4

THE BLOG WRITER SKILL

Save this as `.claude/skills/seo-blog-writer/SKILL.md` in your project. Claude Code automatically detects skills from this folder and triggers them when you say "write a blog post" or "next article".

SAVE AS .CLAUDE/SKILLS/SEO-BLOG-WRITER/SKILL.MD

Frontmatter (between --- markers):

```
name: seo-blog-writer
description: "Write SEO-optimized blog posts for your Next.js site.
Use when the user says 'write a blog', 'next article', 'new post',
or a scheduled agent triggers daily blog writing."
```

Step 1: Pick the Keyword

The user gives you either a specific keyword (e.g., "claude for marketers") or "next" — meaning grab the next unwritten keyword from `references/seo-research/publishing-plan.md`.

Before writing, scan `content/blog/` to make sure you're not duplicating an existing article.

Step 2: Research the Topic

AI changes fast. Before writing a single paragraph, verify your facts with web searches:

- **"[keyword] 2026"** — see what currently ranks, identify your angle
- Every specific AI feature, pricing tier, or capability you plan to mention

One wrong claim ("Claude doesn't have web search") undermines the entire article. Spend 5 minutes checking now to avoid publishing fiction.

Step 3: Plan Before Writing

Identify the cluster your keyword belongs to. Then plan:

- **3-5 internal links** — at least 1 to the cluster's hub page, at least 2 to sibling articles. Vary anchor text.
- **2-3 mid-article CTAs** — subtle callout boxes linking to your community or lead magnet. Helpful tone, not salesy.
- **Search intent** — what does someone Googling this actually want? Answer it in the first 2 paragraphs.

Step 4: Write the Article

Create `content/blog/your-slug.tsx` with the BlogPost export. Content rules:

- Answer search intent in the first 2 paragraphs — Google measures if people bounce back
- 3-6 H2 sections (these auto-populate the table of contents sidebar)
- 2000+ words with real examples, prompt templates people can copy-paste
- No fluff words: "game-changer", "revolutionize", "unlock potential", "delve" are all banned
- Target keyword in: title, first paragraph, 1-2 H2 headings, meta description

Step 5: Wire It Up

The article file alone doesn't make it live. Three things need updating:

1. **lib/blog.ts**
   Add an import for your new post and include it in the allPosts array. Follow the existing pattern.
2. **public/llms-full.txt**
   Add the new article title and URL to the Published articles list. This is how AI models discover and recommend your content.
3. **npm run build**
   Run the build and verify the new route appears with no errors.

Step 6: Report

After everything builds successfully, tell the user: article title and URL path, target keyword, internal links added (list each with anchor text), mid-article CTA placements, build status, and a reminder to submit the URL to Google Search Console.

What's Already Automatic

The site infrastructure handles all of this for every new post — you don't need to touch any of it:

- Sitemap generation (reads from getAllPosts())
- JSON-LD Article schema
- OpenGraph and Twitter meta tags
- Canonical URLs
- Table of contents sidebar (extracted from H2 headings)
- Related articles section (calculated from category and tags)
- Copy buttons on code blocks
- robots.txt (allows all crawlers)

PROMPT 4 OF 4

THE DAILY BLOG AUTOMATION SETUP

Three ways to automate daily blog publishing using Claude Code — from simple scheduling to full CI/CD pipelines. These all use the blog writer skill from Prompt 3.

CHOOSE YOUR AUTOMATION METHOD

Option 1: Claude Code Scheduled Agent (simplest)

In Claude Code, run:

```
/schedule create --name "daily-blog" --cron "0 8 * * *"
--prompt "Use the /seo-blog-writer skill. Pick the next
keyword from the publishing plan and write the article."
```

This runs every morning at 8 AM. It:

1. **Picks the next keyword**
   from the plan
2. **Researches the topic**
3. **Writes the article**
4. **Wires it up**
   (imports, llms.txt, etc.)
5. **Commits**
   (but doesn't push — you review first)

Option 2: n8n Workflow (if you want approval flow)

Create an n8n workflow that:

1. **Triggers on a cron schedule**
   (daily at 8 AM)
2. **Calls Claude Code CLI**
   to write the next article
3. **Creates a GitHub PR**
   instead of committing to main
4. **Sends you a Slack/email notification**
   with the PR link
5. **You review, approve, and merge**

Option 3: GitHub Actions (if you want CI/CD)

Add this to `.github/workflows/daily-blog.yml`:

```
name: Daily Blog Post
on:
  schedule:
    - cron: '0 8 * * 1-5'  # Weekdays at 8 AM
  workflow_dispatch:        # Manual trigger

jobs:
  write-blog:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '24'
      - run: npm install
      - name: Write blog post
        run: npx claude -p "Use the /seo-blog-writer
          skill. Write the next article."
      - name: Create PR
        uses: peter-evans/create-pull-request@v7
        with:
          title: "New blog post: auto-generated"
          body: "Automated daily blog post. Review and merge."
          branch: blog/daily-post
```

The Review Workflow

Regardless of which automation you use, the flow is:

1. **Agent writes the article**
   overnight/morning
2. **You get notified**
   (PR, Slack, or just check the commit)
3. **You review**
   check facts, tone, quality
4. **You say "looks good, push it"**
   or "fix X, Y, Z"
5. **After pushing**
   submit the new URL to Google Search Console for faster indexing

The automation handles the 80% grunt work.
Your 20% is quality control and the final push.

WANT MORE PROMPTS LIKE THESE?

Join 400+ professionals learning to use Claude for real work — workflows, automations, and practical AI skills.

JOIN THE COMMUNITY → skool.com/claude-academy-5969

aitomationacademy.com | youtube.com/@MarcoSudar | skool.com/claude-academy-5969

Disclaimer: We don't guarantee you'll rank #1 on Google or get recommended by AI models. What we can say is that we've seen these results in practice with clients using a setup similar to what's outlined in this guide.